

Utilização de técnicas e ferramentas para aprimorar saídas de IA

Hugo Emílio Nomura, Pedro Henrique Correia de Oliveira, Pedro Henrique Satoru Lima Takahashi, Vitor Monteiro Vianna

Ciência da Computação – Centro Universitário FEI, campus São Paulo

unifhnomura@fei.edu.br, unifpoliveira@fei.edu.br, unifptakahashi@fei.edu.br, unievvianna@fei.edu.br

Orientador: Charles Henrique Porto Ferreira

Departamento de Ciência da Computação, Centro Universitário FEI

cferreira@fei.edu.br

Resumo: Modelos de linguagem generativa frequentemente são utilizados como sistemas diretos de pergunta e resposta. No entanto, resultados recentes indicam que seu desempenho pode ser ampliado quando o modelo base é combinado com técnicas complementares de organização da inferência, decomposição de tarefas e avaliação intermediária da resposta. Este trabalho investiga a hipótese de que a qualidade das saídas produzidas por inteligência artificial depende não apenas do modelo utilizado, mas também da arquitetura que organiza o processo de geração. Para isso, será realizado um estudo experimental comparando o modelo em sua configuração direta com o mesmo modelo operando dentro de um pipeline baseado no *Ralph Loop*, complementado por mecanismos de validação e uso de ferramentas externas quando necessário. A avaliação será conduzida com benchmarks consolidados na literatura, compostos por questões de múltipla escolha distribuídas entre diferentes níveis de dificuldade e áreas do conhecimento. Como se trata de um cenário com respostas objetivas, o desempenho será medido principalmente pela acurácia, com base na comparação entre a alternativa selecionada pelo modelo e o gabarito de cada questão. Dessa forma, espera-se contribuir para a compreensão dos sistemas de IA como composições estruturadas de ferramentas, etapas de processamento e estratégias de orquestração, indo além de mecanismos simples de resposta direta.

Palavras-chaves: inteligência artificial, modelos de linguagem, prompt engineering, orquestração, avaliação experimental, Ralph Loop

I. Introdução

Os modelos de linguagem de grande porte (*Large Language Models* – LLMs) deixaram de ser vistos apenas como mecanismos isolados de geração de texto e passaram a compor sistemas mais amplos de interação, refinamento e coordenação. A literatura mostra que o desempenho desses modelos pode melhorar quando a inferência deixa de ocorrer em uma única chamada e passa a incorporar etapas adicionais de estruturação da tarefa, como instruções exemplificadas, autoformulação de instruções e aprendizagem com dados gerados pelo próprio modelo [1]–[3]. Esse padrão sugere que o valor prático de uma solução de IA não depende somente do modelo base, mas também do conjunto de procedimentos que organiza o caminho entre o prompt e a resposta final.

Esse deslocamento se torna mais evidente quando se observam técnicas recentes de raciocínio e orquestração. O *Chain-of-Thought* mostrou que a explicitação de passos intermediários pode elevar o desempenho em tarefas que exigem inferência [4], enquanto o *Tree of Thoughts* ampliou essa ideia ao tratar a solução como uma busca por estados de pensamento, com geração de alternativas, avaliação heurística e retrocesso [5]. Em paralelo, estudos sobre orquestração dinâmica de prompts e coordenação multiagente indicam que preservação de contexto, consistência lógica e roteamento adaptativo são elementos centrais para tarefas mais complexas [6].

A literatura sobre autoaperfeiçoamento e otimização de prompts reforça essa interpretação. Trabalhos recentes indicam que modelos podem revisar instruções, ajustar formato e melhorar respostas a partir de critérios de utilidade e consistência [2], [3], [7], [8]. Isso ajuda a explicar por que soluções como ChatGPT, Claude e Gemini se comportam, na prática, como plataformas compostas por múltiplas camadas de processamento, e não apenas como um único LLM executando uma pergunta e uma resposta de forma direta. Investigar essas camadas adicionais é, portanto, relevante para compreender o que efetivamente determina a qualidade final das saídas geradas.

Neste trabalho, parte-se da hipótese de que a qualidade de uma mesma tarefa varia conforme a arquitetura de uso do LLM, e não apenas conforme o modelo escolhido. Para testar essa hipótese, será comparado o desempenho do modelo base em sua configuração direta com o desempenho do mesmo modelo operando dentro de um pipeline de orquestração estruturado, baseado no *Ralph Loop*. Embora o método tenha ganhado visibilidade recente na comunidade de desenvolvimento, ainda há pouca literatura acadêmica formal dedicada à sua avaliação sistemática, o que torna sua investigação particularmente relevante no contexto deste estudo. A avaliação será conduzida com benchmarks de múltipla escolha consolidados na literatura, como MMLU e GPQA, nos quais a comparação objetiva entre resposta produzida e gabarito permite medir o impacto real da ar-

quietura proposta [9], [10]. Com isso, busca-se analisar em que medida os ganhos observados em sistemas de IA decorrem da combinação entre modelo, ferramentas e orquestração, e não do modelo isolado.

II. Representação do processo

A comparação entre o uso direto de um modelo de linguagem e o uso do mesmo modelo dentro de uma arquitetura orquestrada constitui o núcleo conceitual deste trabalho. Por essa razão, a representação do processo não deve funcionar apenas como elemento ilustrativo, mas como uma forma de explicitar, desde o início, a hipótese central do estudo: a qualidade da resposta produzida por um LLM pode variar não apenas em função do modelo utilizado, mas também em função da organização da inferência, das etapas intermediárias adotadas e dos mecanismos de controle inseridos entre a entrada e a resposta final [4], [5], [11].

Na configuração direta, a tarefa é enviada ao modelo como uma única instrução, e a resposta é produzida em uma única geração. Essa forma de uso tende a concentrar, em um único passo, operações distintas como interpretação do enunciado, definição da estratégia de resolução, checagem de restrições e escolha da alternativa [1], [12]. Embora essa abordagem seja eficiente para tarefas simples, ela oferece pouco controle sobre o processo intermediário de raciocínio [4]. Já na configuração orquestrada, essas operações são explicitadas e distribuídas em etapas independentes, permitindo maior visibilidade e controle sobre como a resposta é construída [13], [14].

A Figura 1 sintetiza essa distinção em nível macro. Em vez de tratar a resposta como consequência imediata de uma única chamada ao modelo, a arquitetura proposta a representa como resultado de uma sequência organizada de análise, decomposição, processamento, integração e validação [15], [16]. Esse contraste é importante porque estabelece, visualmente, a diferença entre o *baseline* do experimento e a condição principal que será avaliada ao longo do trabalho.

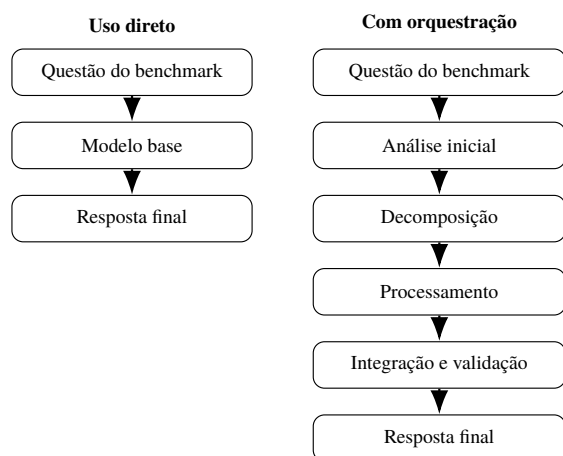


Figura 1. Comparação entre o fluxo direto de inferência e o fluxo orquestrado adotado como arquitetura experimental.

Embora a comparação geral seja suficiente para dife-

renciar as duas condições experimentais, ela ainda não mostra como a arquitetura proposta reorganiza internamente a resolução de uma tarefa. A literatura sobre agentes de linguagem indica que ciclos iterativos de planejamento, execução e reflexão são superiores à geração monolítica em tarefas de múltiplas etapas [5], [13]. Por isso, a Figura 2 detalha a lógica de transformação da questão em resposta, destacando que o processo intermediário não é um acréscimo meramente estético, mas uma estratégia de estruturação da inferência [11], [14]. Nessa perspectiva, a tarefa deixa de ser tratada como um bloco indivisível e passa a ser abordada como uma sequência coordenada de operações menores, cada uma com função específica dentro do pipeline [16].

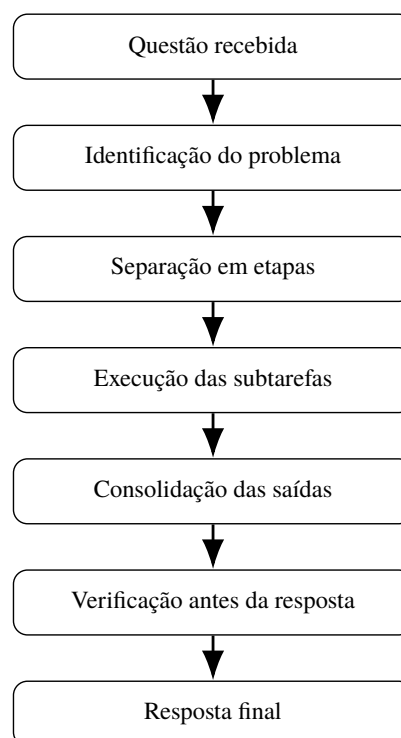


Figura 2. Representação sequencial das etapas que estruturam a passagem da questão à resposta final.

Essa representação torna a seção mais útil ao artigo porque ela deixa de ser apenas uma abertura visual e passa a introduzir, de maneira argumentativa, o objeto metodológico que será aprofundado a seguir. A decomposição em etapas menores está alinhada a resultados experimentais que mostram ganhos de desempenho quando o raciocínio é explicitado antes da escolha da alternativa final [4], [12], [14]. Assim, a representação do processo contribui para reduzir ambiguidades sobre o que exatamente será comparado no experimento e prepara a leitura da metodologia de forma mais coerente.

III. Metodologia

A metodologia deste trabalho foi construída para isolar o efeito da arquitetura de inferência sobre a qualidade das respostas produzidas por um modelo de linguagem. Em vez de comparar modelos distintos entre si, a estratégia central consiste em manter o modelo base constante e alterar

a forma como a tarefa é organizada ao longo da execução [7], [8]. Esse desenho experimental é relevante porque permite analisar se diferenças de desempenho decorrem da capacidade intrínseca do modelo ou da estrutura adotada para conduzir o raciocínio e a produção da resposta final [17], [18].

De modo geral, o experimento compara duas condições principais. A primeira corresponde ao uso direto do modelo, no qual a questão é submetida ao LLM em uma única chamada [12]. A segunda corresponde ao uso do mesmo modelo dentro de um pipeline estruturado, inspirado no *Ralph Loop* [15], em que a resolução da tarefa é distribuída em etapas menores e complementada por verificação intermediária e integração de ferramentas auxiliares [19], [20]. Essa organização foi adotada porque dialoga com a literatura sobre autoaperfeiçoamento iterativo e decomposição de tarefas complexas, que demonstra ganhos quando a inferência incorpora planejamento, revisão e controle parcial [11], [13], [14].

A. Desenho experimental

O desenho experimental parte de uma lógica comparativa rigorosa: a mesma questão será submetida às diferentes condições de execução, preservando-se o mesmo modelo subjacente [9], [10]. Isso permite que a variável principal do estudo seja a arquitetura de processamento, e não a troca do modelo [7]. Em outras palavras, o trabalho busca observar o quanto a organização da inferência interfere no resultado obtido quando o núcleo gerador permanece constante [8], [18].

A Figura 3 resume esse princípio. A partir de uma mesma entrada, duas rotas são construídas: uma rota de referência, mais simples, e uma rota principal, mais estruturada. Ao final, ambas produzem uma resposta comparável com o gabarito da questão, o que permite levar a comparação adiante na etapa de avaliação dos resultados [9], [10].

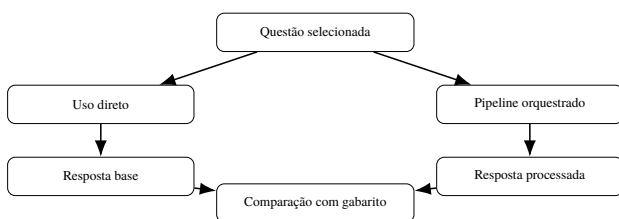


Figura 3. Desenho experimental comparando uso direto e pipeline orquestrado com a mesma base de questões.

Essa forma de organização favorece a interpretação dos resultados porque reduz um problema comum em comparações com LLMs: atribuir ganhos ou perdas ao modelo quando, na realidade, parte do efeito pode estar na maneira como a tarefa foi estruturada [7], [17]. Ao delimitar explicitamente a arquitetura como variável de interesse, o estudo procura oferecer uma análise mais precisa do papel da orquestração na qualidade da resposta [18].

B. Arquitetura proposta

A arquitetura central deste trabalho baseia-se no *Ralph Loop* [15], adotado aqui como uma estratégia de execução em ciclos e etapas mais isoladas, o que reduz a dependência do acúmulo de contexto ao longo do processo. O *Ralph Loop* funciona como um padrão de orquestração cíclico no qual o modelo de linguagem atua de maneira análoga a um agente autônomo, avançando na tarefa por meio de uma iteração contínua entre leitura, planejamento, execução e verificação do contexto atual. Em vez de deduzir a resposta em uma via de mão única, o loop reavalia iterativamente as soluções parciais produzidas, formula os próximos passos necessários e testa sua própria lógica até acumular confiança suficiente de que a tarefa original foi resolvida e a execução pode ser finalizada (*Halt*).

Conceitualmente, essa escolha aproxima o experimento de métodos de raciocínio estruturado e de orquestração deliberada, nos quais resolver um problema exige organizar o fluxo de processamento passo a passo [5], [13], em vez de apenas gerar uma resposta direta após o prompt inicial [12].

Na prática, o pipeline proposto recebe a questão e faz uma análise prévia para identificar o tipo de problema, a área abordada e as restrições da tarefa [16]. Como o enunciado já foi interpretado nessa fase inicial, a resolução passa a ser dividida em ações menores e mais focadas, como: estruturar um plano de raciocínio, levantar as informações necessárias, checar a consistência lógica e validar a alternativa correta. Essa divisão está alinhada à ideia de auto-refinamento iterativo, em que resultados parciais são revisados antes de compor a saída final [11], [14]. As respostas geradas nessas etapas parciais são unidas, podendo ainda passar por um filtro extra de verificação antes de serem registradas [21], [22].

A Figura 4 ilustra essa dinâmica. O objetivo do diagrama não é impor uma regra rígida de execução, mas expor a lógica geral da arquitetura: uma série organizada de ações menores, desenhadas para diminuir a incerteza da inferência e garantir mais controle sobre a resposta gerada [13], [15].

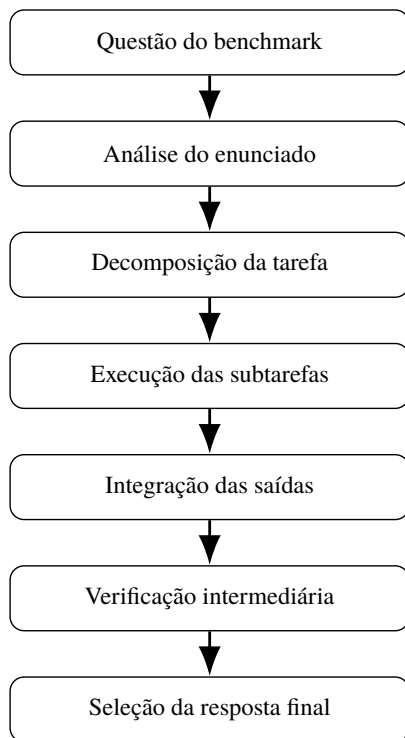


Figura 4. Fluxo da arquitetura proposta, desde a entrada da questão até a consolidação da resposta final.

Essa decomposição tem relevância metodológica porque evita concentrar, em uma única geração, decisões heterogêneas que exigem competências distintas [4], [5]. Em uma questão de física, por exemplo, a identificação das grandezas relevantes não é a mesma operação da escolha final da alternativa correta; em uma questão de filosofia, reconhecer a tese central também não equivale à comparação entre alternativas. Ao explicitar essas diferenças em etapas menores, o pipeline procura alinhar a execução do modelo a uma lógica de resolução mais estruturada [11], [18].

C. Componentes auxiliares

Embora a arquitetura principal já seja suficiente para estabelecer a comparação central do artigo, o pipeline poderá incorporar componentes auxiliares voltados ao fortalecimento da resposta final. Entre esses componentes, destacam-se mecanismos de verificação intermediária, nos quais uma saída parcial pode ser inspecionada antes de ser aceita como resposta consolidada [21], [22], e ferramentas externas de apoio integradas via protocolos estruturados [19], [20]. Esses elementos não constituem o objeto principal da hipótese experimental, mas ampliam a capacidade de controle e fundamentam-se em evidências recentes de que agentes com acesso a ferramentas superam modelos isolados em tarefas que exigem consultas externas [16], [23].

A Figura 5 apresenta uma visão mais estrutural do pipeline, destacando a relação entre o núcleo de execução, o módulo de verificação e os recursos auxiliares. Essa representação evidencia que a arquitetura proposta não é apenas uma cadeia linear de passos, mas um arranjo em que diferentes componentes apoiam a produção e a validação

da resposta [13], [15].

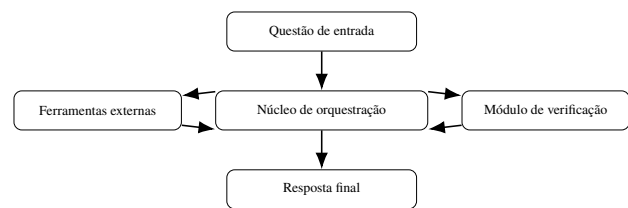


Figura 5. Visão estrutural da arquitetura proposta, com o núcleo de orquestração interagindo com módulos de ferramentas e verificação.

A inserção desses módulos auxiliares ajuda a explicar por que a arquitetura proposta deve ser compreendida como uma composição de etapas e não apenas como um prompt maior ou mais detalhado. O ganho esperado, caso ele exista, não decorre exclusivamente da redação da instrução inicial, mas da possibilidade de distribuir funções diferentes entre partes distintas do sistema, permitindo revisão, controle e apoio externo durante a execução [11], [19], [20].

D. Controle da execução

Para que a comparação entre as condições experimentais seja válida, a execução deverá seguir um protocolo uniforme. Cada questão selecionada será submetida individualmente às condições definidas no estudo, com registro da alternativa produzida, do contexto de execução e da condição aplicada [9], [10]. Esse cuidado é importante porque a utilidade do experimento não depende apenas da existência de um pipeline mais elaborado, mas da possibilidade de comparar as respostas obtidas sob critérios consistentes [7], [8].

Além disso, o protocolo prevê que a arquitetura proposta seja analisada não somente em termos de acerto final, mas também em termos de comportamento por etapa [21], [22]. Ainda que a métrica principal seja a acurácia, a própria organização do pipeline permite observar em que ponto a estrutura contribui mais para a robustez da resposta, seja na interpretação inicial, na decomposição da tarefa, na integração das saídas ou na validação da alternativa escolhida [13], [14]. Com isso, a metodologia não se limita a descrever um sistema, mas prepara o terreno para uma avaliação mais informativa na seção seguinte.

IV. Avaliação dos resultados

A avaliação dos resultados exige um protocolo que seja objetivo, reproduzível e comparável com referências já consolidadas na literatura. A escolha de benchmarks de múltipla escolha com gabarito fixo atende a esse requisito porque permite mensurar o desempenho por meio de respostas objetivas, reduzindo ambiguidades de interpretação e facilitando a comparação entre diferentes condições experimentais [9], [10]. Esse tipo de estrutura também favorece a replicabilidade do experimento, já que a resposta correta de cada questão é previamente conhecida e o critério principal de avaliação pode ser expresso diretamente em termos de acurácia [7], [8].

A. Benchmarks adotados

Os benchmarks utilizados na avaliação serão selecionados a partir de conjuntos já consolidados na literatura, com destaque para MMLU e GPQA [9], [10]. O MMLU oferece ampla diversidade temática, cobrindo áreas como humanidades, ciências sociais, STEM e medicina [9], enquanto o GPQA introduz questões mais difíceis e especializadas em biologia, química e física [10]. A combinação entre esses benchmarks permite equilibrar amplitude e profundidade na análise, evitando que a avaliação dependa de apenas um tipo de tarefa ou de uma única faixa de dificuldade.

Além da variedade de domínios, a seleção buscará contemplar diferentes níveis de complexidade, incluindo questões compatíveis com ensino médio, graduação, pós-graduação e doutorado, conforme a natureza de cada benchmark e de cada tópico escolhido. Essa diversidade é relevante porque permite observar se a arquitetura proposta mantém comportamento consistente ao longo de diferentes graus de exigência cognitiva. Assim, a avaliação deixa de medir apenas desempenho médio e passa a considerar também a robustez do sistema diante de tarefas com perfis distintos [9], [10].

B. Amostragem das questões

Em vez de utilizar integralmente todos os itens disponíveis em cada benchmark, o experimento trabalhará com uma seleção amostral representativa de questões. O dimensionamento exato dessa amostra e a definição dos tópicos específicos serão estabelecidos de modo a equilibrar a viabilidade computacional e a diversidade temática necessária para a análise. Os tópicos avaliados deverão contemplar domínios como matemática, física, biologia, linguagens, filosofia e outros conjuntos equivalentes que se mostrem adequados ao recorte do experimento.

Essa estratégia de amostragem busca preservar representatividade sem tornar a execução excessivamente extensa ou custosa. Ao organizar a avaliação por tópicos e por níveis de dificuldade, torna-se possível examinar se o impacto da arquitetura proposta varia conforme o tipo de conhecimento exigido ou conforme a complexidade da questão. Dessa forma, a análise poderá identificar padrões mais específicos do que os oferecidos por uma única taxa global de acerto.

C. Procedimento de avaliação

Cada questão selecionada será submetida individualmente às condições experimentais definidas neste trabalho. A condição de referência corresponde ao uso direto do modelo base, sem modificações adicionais. A condição principal corresponde ao uso do mesmo modelo dentro do pipeline baseado no *Ralph Loop*, juntamente com as adaptações e componentes auxiliares que forem efetivamente empregados na implementação final, como mecanismos de verificação intermediária ou ferramentas externas de apoio [15], [19]–[22]. De forma complementar, um modelo mais avançado poderá ser avaliado em sua configuração direta, funcionando como referência adicional para situar os resul-

tados obtidos.

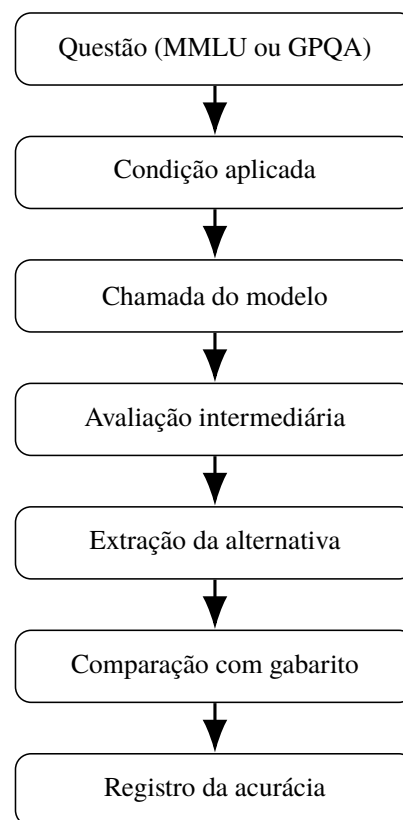


Figura 6. Pipeline de avaliação experimental aplicado a cada questão dos benchmarks.

A resposta final produzida em cada execução será comparada com o gabarito correspondente, permitindo o cálculo da acurácia por condição experimental. Além da taxa de acerto global, os resultados poderão ser organizados por benchmark, por área temática e por nível de dificuldade, o que torna a análise mais detalhada e mais compatível com o objetivo de verificar em quais contextos a arquitetura proposta apresenta maior impacto. Esse procedimento mantém o mesmo critério de decisão para todas as condições avaliadas e, por isso, preserva a objetividade da comparação [7], [23].

D. Critérios de análise

A análise dos resultados será centrada na comparação entre o desempenho do modelo base e o desempenho do pipeline proposto, tomando a acurácia como métrica principal. A interpretação poderá considerar recortes como variação entre benchmarks, comportamento por área temática, diferenças entre níveis de dificuldade e distância entre os resultados obtidos pelo modelo base, pelo pipeline orquestrado e pelo modelo adicional de referência, caso este seja incluído na etapa experimental.

Essa organização permite que a discussão vá além de uma simples média agregada de acertos. Caso o pipeline apresente ganhos mais pronunciados em determinadas áreas ou em níveis específicos de complexidade, esses padrões poderão ser descritos com maior precisão. Da mesma forma, se os efeitos observados forem modestos ou heterogêneos, a estrutura adotada também permitirá identificar

com clareza em quais contextos a arquitetura proposta se mostra mais ou menos eficaz.

V. Referências

- [1] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever e D. Amodei, “Language models are few-shot learners,” em *Advances in Neural Information Processing Systems*, 2020.
- [2] Y. Wang, Y. Kordi, S. Mishra, A. Liu, N. A. Smith, D. Khashabi e H. Hajishirzi, “Self-instruct: Aligning language models with self-generated instructions,” em *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2023.
- [3] J. Huang, S. S. Gu, L. Hou, Y. Wu, X. Wang, H. Yu e J. Han, “Large language models can self-improve,” em *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [4] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le e D. Zhou, “Chain-of-thought prompting elicits reasoning in large language models,” *ArXiv preprint arXiv:2201.11903*, 2022.
- [5] S. Yao, D. Yu, J. Zhao, I. Shafran, T. L. Griffiths, Y. Cao e K. Narasimhan, “Tree of thoughts: Deliberate problem solving with large language models,” *ArXiv preprint arXiv:2305.10601*, 2023.
- [6] Unknown, “Reasoning-aware prompt orchestration for multi-agent language model systems,” *Manuscript / preprint*, 2025.
- [7] L. Zhang, K. Jijo, S. Setty, E. Chung, F. Javid, N. Vidra e T. Clifford, “Enhancing large language model performance to answer questions and extract information more accurately,” *ArXiv preprint arXiv:2402.01722*, 2024.
- [8] Y. Liu, J. Xu, L. L. Zhang, Q. Chen, X. Feng, Y. Chen, Z. Guo, Y. Yang e P. Cheng, “Beyond prompt content: Enhancing llm performance via content-format integrated prompt optimization,” *ArXiv preprint arXiv:2502.04295*, 2025.
- [9] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song e J. Steinhardt, “Measuring massive multitask language understanding,” em *International Conference on Learning Representations*, 2021.
- [10] D. Rein, B. L. Hou, A. C. Stickland, J. Petty, R. Y. Pang, J. Dirani, J. Michael e S. R. Bowman, “Gpqa: A graduate-level google-proof q&a benchmark,” *ArXiv preprint arXiv:2311.12022*, 2023.
- [11] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabh-moye, Y. Yang, S. Welleck, B. Clark, A. Bosse-lut, X. Ren, M. Fazel-Zarandi, A. Parikh, H. Ha-jishirzi, K. McKeown e P. Clark, “Self-refine: Iterative refinement with self-feedback,” *ArXiv preprint arXiv:2303.17651*, 2023.
- [12] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo e Y. Iwa-sawa, “Large language models are zero-shot reasoners,” *ArXiv preprint arXiv:2205.11916*, 2022.
- [13] N. Shinn, F. Cassano, B. Labash, A. Gopinath, K. Narasimhan e S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *ArXiv preprint arXiv:2303.11366*, 2023.
- [14] X. Wang, J. Wei, D. Schuurmans, Q. Le, E. Chi, S. Narang, A. Chowdhery e D. Zhou, “Self-consistency improves chain of thought reasoning in language models,” *ArXiv preprint arXiv:2203.11171*, 2023.
- [15] G. Ghuntley, *Ralph loop: A structured orchestration pattern for llm inference*, Online. Disponível em: <https://ghuntley.com/specs/ralph/>, Acesso em: abr. 2026, 2025.
- [16] Y. Shen, K. Song, X. Tan, D. Li, W. Lu e Y. Zhuang, “Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face,” em *Advances in Neural Information Processing Systems*, 2023.
- [17] OpenAI, “Gpt-4 technical report,” *ArXiv preprint arXiv:2303.08774*, 2023.
- [18] A. Huttula, “Advanced prompt engineering: Systematic approaches to enhance llm performance,” tese de mestrado, Jamk University of Applied Sciences, 2025.
- [19] Anthropic, “Model context protocol (mcp): Enabling tool use and external memory in language model pipelines,” *Technical Report*, 2025, Disponível em: <https://modelcontextprotocol.io>.
- [20] A. Sarkar, P. Gupta e R. Mehta, “Integrating external tools with large language models via structured protocols,” *ArXiv preprint arXiv:2503.11200*, 2025.
- [21] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez e I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” em *Advances in Neural Information Processing Systems*, 2024.
- [22] D. Li, B. He, Y. Li, L. Huang, Y. Wang, Z. Xu e H. Xu, “From generation to judgment: Opportunities and challenges of llm-as-a-judge,” *ArXiv preprint arXiv:2411.16594*, 2024.
- [23] Y. Chen, J. Chen, R. Meng, J. Yin, N. Li, C. Fan, C. Wang, T. Pfister e J. Yoon, “Tumix: Multi-agent test-time scaling with tool-use mixture,” *ArXiv preprint arXiv:2510.01279*, 2025.